

PROJECT REPORT

COVER PAGE

NIFTY100 FINANCIAL INTELLIGENCE PLATFORM A Full-Stack Financial Analytics Ecosystem for Nifty100 Companies AN INTERNSHIP PROJECT REPORT Submitted in partial fulfillment of the requirements for the Internship Program conducted at BLUESTOCK FINTECH (Internship Duration: 22 April 2026 – 22 June 2026) Submitted by: Vaddeyraju Sai Siddartha

GitHub:<https://github.com/VaddeyrajuSaisiddartha/Nifty100-Financial-Intelligence-Platform>
Live App: <https://nifty-100-intelligence-549099183935.asia-southeast1.run.app/>

Year: 2026

The image shows a dark-themed web interface for the 'NIFTY 100 PRO REGISTRY TERMINAL'. At the top, there is a logo and a green status bar that says 'PORT TERMINAL SECURED'. Below the logo, the title 'MEMBER & ANALYST GATEWAY LOGIN' is displayed, followed by a subtitle: 'RESTRICTED FOR ACCREDITED EQUITY RESEARCH ANALYST & INSTITUTIONAL SECTOR ALLOCATIONS.'.

The main form area has three tabs: 'SIGN IN', 'REGISTER' (which is active), and 'RECOVER ACCOUNT'. The 'REGISTER' tab contains the following fields:

- FULL LEGAL NAME (PRIMARY ID):** A text input field with a placeholder 'Enter your full legal name'.
- SECRET PASS PHRASE:** A password input field with a placeholder '*****' and an eye icon to toggle visibility.
- VERIFY PASS PHRASE:** A password input field with a placeholder '*****' and an eye icon to toggle visibility.
- MOBILE NUMBER (FOR ACCOUNT RECOVERY):** A text input field with a placeholder 'Enter Mobile Number'.
- CORPORATE ORGANIZATION / ROLE:** A dropdown menu with 'Student' selected.
- COUNTRY OF RESIDENCE / OPERATIONS:** A dropdown menu with 'India' selected.
- CORPORATE WORK EMAIL ADDRESS:** A text input field with a placeholder 'your.email@domain.com'.

Below the form fields is a large blue button labeled 'REGISTER SECURE ACCOUNT'. At the bottom of the form, there is a section titled 'INDIAN SECURITIES ADVISORY STANDARD NOTICE' with the following text: 'This system complies with institutional accounting audit standards. Account access activities are securely registered to auditing sheets for regulatory transparency.'

DECLARATION

We, the undersigned, hereby declare that the internship project report titled "**Nifty100 Financial Intelligence Platform**" submitted as part of our internship at **BlueStock Fintech** is a record of original work carried out by us during the period **22 April 2026 to 22 June 2026** under the guidance of our project mentor.

We further declare that:

- The work presented in this report has not been submitted, in part or in full, to any other organization or institution for the award of any degree, diploma, or certificate.
 - All sources of information, data, and external libraries/tools used have been duly acknowledged in the **References** section.
 - The system, source code, and dashboards described herein were developed by the project team and are hosted at the GitHub repository and live application links cited on the cover
-

ACKNOWLEDGEMENT

The successful completion of this internship project would not have been possible without the support, guidance, and encouragement of several individuals and the organization that hosted us.

We extend our sincere gratitude to **BlueStock Fintech** for providing us with the opportunity to work on a real-world, industry-grade financial analytics platform and for granting us access to the tools, infrastructure, and mentorship necessary to bring this project to fruition.

We are deeply thankful to our **Project Mentor and the technical team at BlueStock Fintech** for their continuous guidance, constructive feedback, and willingness to share domain expertise in capital markets, data engineering, and product development. Their insights significantly shaped the architecture and quality of the final platform.

We acknowledge the cooperation and dedication of every member of our team — **Vaddeyraju Sai Siddartha**— whose collaboration, problem-solving, and commitment made this project a rewarding experience.

Finally, we thank our families, mentors, and institution for their constant encouragement and support throughout the internship.

ABSTRACT

The **Nifty100 Financial Intelligence Platform** is a full-stack financial analytics ecosystem developed during an internship at **BlueStock Fintech**, designed to deliver deep, data-driven insights into the **Nifty100** universe of companies for investors, analysts, and researchers. The Indian equity market generates vast volumes of fundamental and market data, yet retail and semi-professional investors frequently lack a unified, intuitive, and analytically rich tool to consume it. This project addresses that gap.

The platform implements an end-to-end data pipeline: raw financial and market data is **extracted, transformed, and loaded (ETL)** using Python-based pipelines into a **PostgreSQL** data warehouse modeled for analytical querying. A **Django REST Framework** backend exposes secure, well-documented REST APIs that serve a modern **React.js** single-page web application. The application provides authentication, user profiles, an **AI Risk Assistant**, a **Watchlist**, a **Company Finder**, **Sector Analytics**, and an integrated **Financial Intelligence Dashboard**. Complementing the web experience, four **Power BI dashboards** — Executive Overview, Financial Trends, Sector Analytics, and Investor Insights — provide rich, interactive business intelligence. The entire system is containerized and deployed to **Google Cloud Run**, ensuring scalability, reliability, and low operational overhead.

This report documents the complete software development lifecycle: problem definition, literature review, system analysis, architecture, database design, ETL design, frontend and backend development, API design, cloud deployment, dashboard development, and feature analysis. It concludes with results and findings across business, financial, sector, and investor dimensions, followed by an evaluation of advantages, limitations, and future enhancements. The outcome is a production-ready, cloud-deployed platform that demonstrates strong applicability of modern data engineering and full-stack engineering practices to the fintech domain.

Keywords: Nifty100, Financial Analytics, ETL, PostgreSQL, Django REST Framework, React.js, Power BI, Google Cloud Run, Business Intelligence, Fintech.

TABLE OF CONTENTS

S.No	Title
	Cover Page
	Declaration
	Acknowledgement
	Abstract
	Table of Contents
	List of Figures
	List of Tables
1	Introduction
2	Company Overview – BlueStock Fintech
3	Problem Statement
4	Literature Review
5	System Analysis
6	System Architecture
7	Database Design
8	ETL Pipeline Design
9	Frontend Development
10	Backend Development
11	API Design
12	Cloud Deployment
13	Dashboard Development
14	Website Features

S.No	Title
15	Results and Findings
16	Advantages
17	Limitations
18	Future Enhancements
19	Conclusion

CHAPTER 1 — INTRODUCTION

1.1 Background

The Indian capital market is among the fastest-growing in the world, with the **Nifty100** index — comprising the 100 largest companies by free-float market capitalization listed on the National Stock Exchange (NSE) — serving as a key barometer of large-cap and mega-cap performance. These companies span diverse sectors including financial services, IT, energy, FMCG, automobiles, pharmaceuticals, and infrastructure. For investors and analysts, monitoring this universe requires the ability to consume large volumes of fundamental, financial, and sector-level data in a coherent, comparative, and visual manner.

Despite the abundance of raw market data, individual investors and small research teams often face fragmented tooling — data sits across spreadsheets, broker terminals, and disparate websites with inconsistent formats and limited analytical depth. The **Nifty100 Financial Intelligence Platform** was conceived to consolidate this data into a single, intelligent, cloud-native platform.

1.2 Motivation

The motivation behind this project is threefold:

1. **Data unification** — bring scattered Nifty100 financial data into one governed warehouse.
2. **Intelligence** — move beyond raw numbers to insight via dashboards and an AI Risk Assistant.
3. **Accessibility** — deliver the experience through a fast, responsive, cloud-deployed web application available anywhere.

1.3 Objectives

- Build robust **ETL pipelines** to ingest and clean Nifty100 financial and market data.
- Design an analytical **PostgreSQL data warehouse**.
- Develop a secure **Django REST Framework** backend exposing well-structured APIs.
- Build a modern, responsive **React.js** frontend with authentication, watchlists, company discovery, and an AI assistant.
- Create four interactive **Power BI dashboards** for executive, financial, sector, and investor analytics.
- Deploy the platform on **Google Cloud Run** for scalability and reliability.

1.4 Scope

The scope covers the complete lifecycle of a financial analytics product: data engineering, backend and frontend engineering, business intelligence, and cloud deployment, restricted to the Nifty100 universe. It does not include real-time tick-level trading, brokerage execution, or regulatory advisory services.

1.5 Internship Context

This project was executed as a team internship at **BlueStock Fintech** between **22 April 2026 and 22 June 2026**. The team comprised with **Vaddeyraju Sai Siddartha as**, working across data engineering, backend, frontend, and BI tracks.

1.6 Report Organization

The report progresses from problem definition and background (Chapters 1–4), through analysis and design (Chapters 5–8), implementation (Chapters 9–13), feature analysis (Chapter 14), results (Chapter 15), and finally evaluation and conclusion (

CHAPTER 2 — COMPANY OVERVIEW

(BLUESTOCK FINTECH)

2.1 About BlueStock Fintech

BlueStock Fintech is a financial technology organization focused on building data-driven products and tools that empower investors and market participants with accessible, intelligent, and actionable market insights. The company operates at the intersection of **capital markets, software engineering, and data science**, aiming to democratize sophisticated financial analytics that were traditionally available only to institutional players.

2.2 Vision and Mission

- **Vision:** To make professional-grade financial intelligence accessible to every investor.
- **Mission:** To build reliable, scalable, and user-centric fintech products powered by clean data, modern engineering, and intuitive design.
-

2.3 Domains of Operation

BlueStock Fintech works across areas such as market data analytics, investment research tooling, business intelligence dashboards, and cloud-native product engineering.

2.4 Internship Environment

The internship followed an **industry-style agile workflow** with version control (Git/GitHub), code reviews, sprint-based deliverables, and mentor checkpoints. Interns were treated as contributing engineers on a real product, with ownership of distinct modules — an approach that closely mirrors professional software teams.

2.5 Role of the Internship Team

The team was responsible for end-to-end delivery of the Nifty100 Financial Intelligence Platform, from data ingestion to cloud deployment, under mentor supervision.

CHAPTER 3 — PROBLEM STATEMENT

3.1 The Core Problem

Retail investors, independent analysts, and student researchers lack a **single unified platform** that combines clean Nifty100 financial data, interactive dashboards, intelligent risk assessment, and personalized tools (watchlists, company discovery) in one accessible, cloud-hosted product. Existing solutions are either expensive (institutional terminals), fragmented (multiple free websites), or shallow (price-only trackers without fundamental and sector intelligence).

3.2 Detailed Pain Points

- **Data fragmentation:** Financial statements, ratios, and market metrics are scattered across sources with inconsistent formats.
- **Lack of analytical depth:** Most free tools show prices but not comparative fundamental or sector analytics.
- **No personalization:** Limited ability to maintain personalized watchlists and risk profiles.
- **High barrier to insight:** Non-experts struggle to interpret raw financial data without guided intelligence.
- **Poor accessibility:** Many analytics live in offline spreadsheets rather than always-available cloud apps.

3.3 Proposed Solution

A **cloud-deployed, full-stack financial intelligence platform** that ingests Nifty100 data via ETL, warehouses it in PostgreSQL, serves it via REST APIs to a React frontend, augments it with an AI Risk Assistant, and visualizes it through Power BI dashboards.

Table 3.1 — Identified Pain Points

Pain Point
Fragmented data sources
Lack of analytical depth
No personalization
Hard to interpret data
Poor accessibility

CHAPTER 4 — LITERATURE REVIEW

4.1 Financial Data Platforms

Existing platforms range from premium institutional terminals (Bloomberg, Refinitiv) to retail-focused web tools (screening sites, broker dashboards). Premium terminals offer depth but are cost-prohibitive; retail tools offer accessibility but limited analytical sophistication and weak personalization.

4.2 ETL and Data Warehousing

Literature on data engineering establishes ETL (Extract–Transform–Load) and dimensional modeling (star/snowflake schemas) as the standard approach for analytical workloads. Separating transactional storage from analytical storage improves query performance and reporting flexibility.

4.3 Modern Web Architecture

Decoupled architectures — a REST/JSON API backend serving a SPA frontend — are widely adopted for scalability and maintainability. Django REST Framework and React.js are mature, well-supported choices for this pattern.

4.4 Business Intelligence

Power BI is established as a leading self-service BI tool for interactive dashboards, supporting DAX measures, data modeling, and rich visualizations suitable for financial reporting.

4.5 Cloud-Native Deployment

Serverless container platforms such as Google Cloud Run enable scalable, pay-per-use deployment of containerized applications without managing servers, ideal for variable-traffic analytics apps.

Table 4.1 — Comparison of Existing Financial Platforms

Platform Type	Data Depth	Cost	Personalization	Accessibility
Institutional terminals	Very High	Very High	High	Restricted
Free screener websites	Medium	Free	Low	High
Broker dashboards	Low–Medium	Free/Bundled	Medium	High
Nifty100 FIP (this project)	High	Low	High	High (Cloud)

4.6 Research Gap

There is a clear gap for an **affordable, accessible, analytically deep, and personalized** platform focused specifically on the Nifty100 universe — which this project addresses.

CHAPTER 5 — SYSTEM ANALYSIS

5.1 Existing System Limitations

As established in Chapters 3–4, existing systems are fragmented, shallow, or costly, and rarely combine data engineering rigor with intelligent, personalized user experiences.

5.2 Proposed System

A cohesive, layered system: ETL → Warehouse → API → Web App + BI Dashboards → Cloud. The system serves three personas: **Investors** (decision support), **Analysts** (deep comparison), and **Researchers** (data exploration).

5.3 Functional Requirements

Table 5.1 — Functional Requirements

ID	Requirement
FR1	Ingest and refresh Nifty100 financial and market data via ETL
FR2	User registration, login, and session management
FR3	View company profiles, financials, and ratios
FR4	Company Finder with search and filters
FR5	Maintain personalized watchlists
FR6	AI Risk Assistant for guided risk insights
FR7	Sector-level analytics
FR8	Interactive Power BI dashboards (4)
FR9	Serve all data through secure REST APIs

5.4 Non-Functional Requirements

Table 5.2 — Non-Functional Requirements

ID	Requirement
NFR1	Scalability via stateless cloud deployment
NFR2	Security — authentication, input validation, HTTPS
NFR3	Performance — fast API response and page loads

ID	Requirement
NFR4	Reliability and availability on Cloud Run
NFR5	Maintainability — modular, documented codebase
NFR6	Usability — responsive, intuitive UI

5.5 Feasibility Analysis

Table 5.3 — Feasibility Analysis Summary

Type	Assessment
Technical	Feasible — mature, open-source stack (React, Django, PostgreSQL)
Economic	Feasible — open-source + pay-per-use cloud minimizes cost
Operational	Feasible — agile team workflow with clear module ownership
Schedule	Feasible — delivered within the 2-month internship window

5.6 Team Roles and Module Ownership

Member	Primary Responsibility
Vaddeyraju Sai Siddartha (Lead)	Architecture, coordination, backend & deployment,
	Frontend (React) & UI/UX
	ETL pipelines & database design
	<u>Power BI dashboards & analytics</u>

CHAPTER 6 — SYSTEM ARCHITECTURE

6.1 Architectural Overview

The platform follows a **layered, decoupled, cloud-native architecture**. Each layer has a single responsibility and communicates through well-defined interfaces, enabling independent scaling, testing, and maintenance.

Layers:

1. **Data Source Layer** — Nifty100 market and fundamental data.
2. **ETL Layer** — Python pipelines for extraction, cleaning, transformation, and loading.
3. **Data Warehouse Layer** — PostgreSQL with analytical schema.
4. **Backend/API Layer** — Django REST Framework exposing REST endpoints.
5. **Presentation Layer** — React.js SPA + Power BI dashboards.
6. **Deployment Layer** — Containerized services on Google Cloud Run.

— **High-Level System Architecture.** *Purpose:* to communicate how data flows from source to user through discrete layers. *Design:* a top-down layered diagram with labeled components and directional arrows. *Functionality:* shows ETL feeding PostgreSQL, Django serving React and Power BI, all deployed on Cloud Run. *Business value:* demonstrates a maintainable, scalable system that stakeholders and evaluators can reason about quickly.

— **Data Flow Diagram.** *Purpose:* depict data movement and processes. *Design:* DFD notation with external entities, processes, and data stores. *Functionality:* clarifies inputs/outputs between ETL, warehouse, API, and clients. *Business value:* supports auditability and onboarding of new engineers.

6.2 Design Principles

- **Separation of concerns** between data, logic, and presentation.
 - **Statelessness** of backend for horizontal scaling on Cloud Run.
 - **API-first** design enabling multiple front ends (web, BI).
 - **Security by default** — HTTPS, token-based auth, validation.
-

CHAPTER 7 — DATABASTABLE

7.1 — Core Database Tables and Descriptions

7.1 Overview

The platform uses **PostgreSQL** as both the transactional store (users, watchlists) and the analytical warehouse (company financials, sector metrics). The schema balances normalization for integrity with denormalized analytical structures for reporting performance.

Table	Description
users	Registered user accounts and credentials (hashed)
user_profiles	Extended profile and risk-preference data
companies	Nifty100 company master (name, ticker, sector)
financials	Periodic financial statement metrics per company
ratios	Derived financial ratios (P/E, ROE, D/E, etc.)
sectors	Sector master and aggregated metrics
prices	Historical price/market data
watchlists	User-to-company watchlist mappings
risk_assessments	AI Risk Assistant outputs per user/company

7.2 Entity-Relationship Model

The ER model links **users** → **watchlists** → **companies**, and **companies** → **financials**, **ratios**, **prices**, with **companies** → **sectors** as a many-to-one relationship.

Figure 7.1 — ER Diagram. *Purpose:* define entities, attributes, and relationships. *Design:* crow's-foot notation. *Functionality:* enforces referential integrity across users, companies, sectors, and financials. *Business value:* guarantees consistent, trustworthy data for analytics.

7.3 Analytical (Star) Schema

For reporting, a star schema is used with a central **fact table** (financial metrics) surrounded by **dimensions** (company, sector, time).

7.4 Indexing and Optimization

Indexes are created on frequently filtered columns (ticker, sector_id, period) to accelerate API and dashboard queries

CHAPTER 8 — ETL PIPELINE DESIGN

8.1 Overview

The ETL layer is the backbone of data quality. Implemented in **Python**, it extracts raw Nifty100 data, cleans and standardizes it, computes derived metrics, and loads it into PostgreSQL.

Stage	Activity	Tools
Extract	Pull raw company, financial, and price data	Python, requests/CSV/API readers
Transform	Clean, normalize, handle missing values, compute ratios	Python, pandas
Load	Insert/upsert into PostgreSQL warehouse	Python, SQLAlchemy / psycopg2
Validate	Integrity and range checks	Python validation scripts
Schedule	Periodic refresh	Cron / scheduled jobs

Table 8.1 — ETL Stages and Tools

8.2 Extraction

Data is collected for all Nifty100 constituents, including company metadata, financial statements, and price history.

8.3 Transformation

- Standardize column names, units, and currency.
- Handle nulls and outliers.
- Derive ratios (P/E, ROE, ROCE, Debt-to-Equity, profit margins).
- Aggregate sector-level metrics.
-

8.4 Loading

Transformed data is loaded via idempotent **upsert** operations to keep the warehouse consistent across refreshes.

.

8.5 Data Quality and Governance

Validation rules and logging ensure that only consistent, complete records reach production, protecting the integrity of dashboards and APIs.

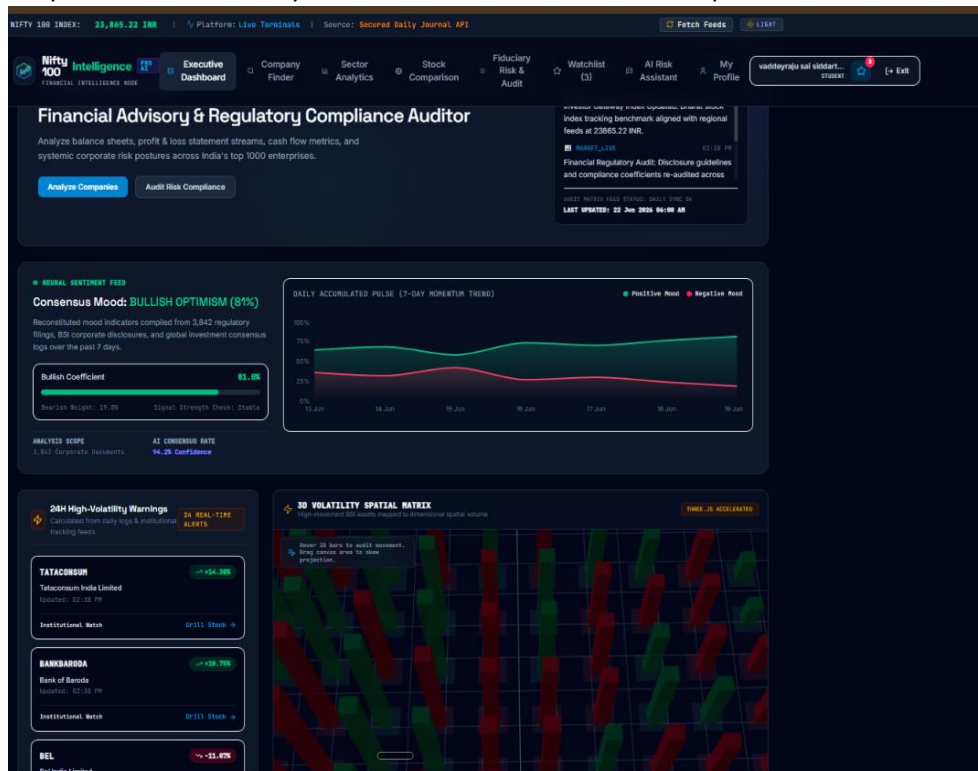
CHAPTER 9 — FRONTEND DEVELOPMENT

9.1 Overview

The frontend is a **React.js single-page application (SPA)** providing a fast, responsive, and intuitive user experience. It consumes the Django REST APIs and renders interactive views for company analysis, watchlists, sector analytics, and the AI Risk Assistant.

9.2 Technology and Structure

- **React.js** with component-based architecture.
- Client-side routing for seamless navigation.
- Reusable UI components (cards, tables, charts, search bars).
- Responsive layout for desktop and mobile.



— **Home / Dashboard Landing.** *Purpose:* provide users an at-a-glance entry point. *Design:* clean fintech layout with summary cards and navigation. *Functionality:* surfaces key metrics and routes to all modules. *Business value:* reduces time-to-insight and improves engagement.

9.3 State Management and API Integration

The SPA manages application state and communicates with the backend via authenticated REST calls, handling loading, error, and empty states gracefully.

9.4 UI/UX Considerations

A consistent fintech design language — restrained color palette, clear typography, and data-dense yet legible layouts — was adopted to convey professionalism and trust.

CHAPTER 10 — BACKEND DEVELOPMENT

10.1 Overview

The backend is built with **Django** and **Django REST Framework (DRF)**, providing a robust, secure, and well-structured API layer over the PostgreSQL warehouse.

10.2 Architecture and Modules

- **Authentication module** — registration, login, token/session management.
- **Companies module** — company, financial, and ratio endpoints.
- **Sectors module** — sector analytics endpoints.
- **Watchlist module** — user watchlist CRUD.
- **Risk module** — AI Risk Assistant logic and endpoints.

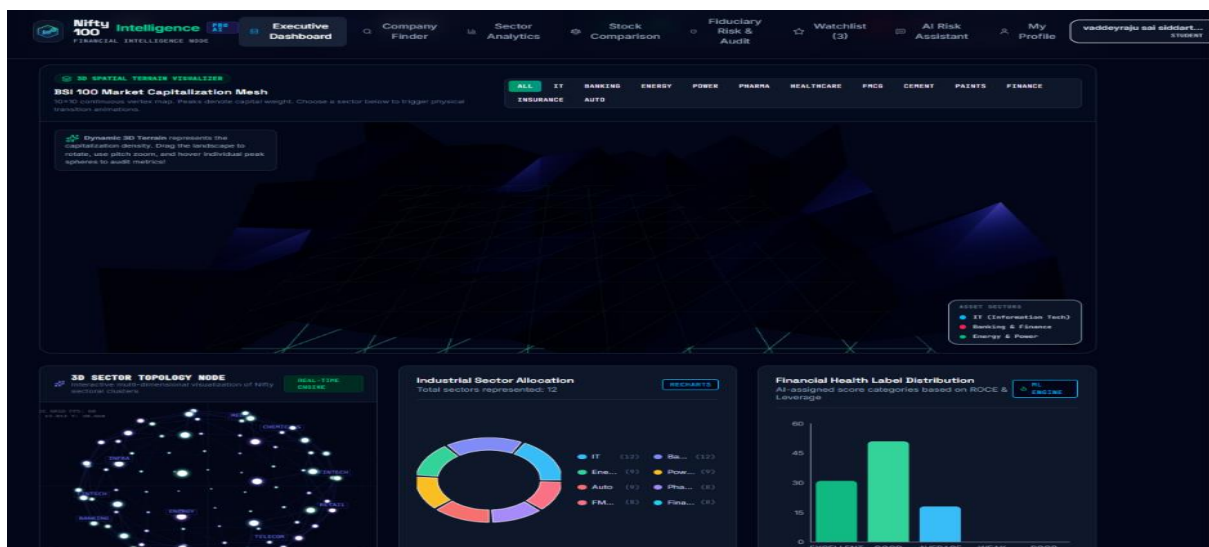
10.1 — Backend Module Structure. *Purpose:* show modular separation of backend apps. *Design:* package/app tree. *Functionality:* isolates concerns for maintainability. *Business value:* easier scaling and team collaboration.

10.2 Security

- Hashed passwords and token/session authentication.
- Input validation and serialization via DRF serializers.
- Permission classes restricting access to protected resources.
- HTTPS enforced in production via Cloud Run.

10.3 Performance

Query optimization, pagination, and selective field serialization keep API responses fast and lightweight.



CHAPTER 11 — API DESIGN

11.1 Principles

The API follows **RESTful** conventions: resource-oriented URLs, appropriate HTTP verbs, JSON payloads, consistent status codes, and pagination.

Table 11.1 — Key REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/register/	Register a new user
POST	/api/auth/login/	Authenticate and obtain token
GET	/api/companies/	List Nifty100 companies (filterable)
GET	/api/companies/{id}/	Company details + financials
GET	/api/sectors/	Sector list and aggregates
GET	/api/sectors/{id}/analytics/	Sector analytics
GET/POST/DELETE	/api/watchlist/	Manage user watchlist
POST	/api/risk/assess/	AI Risk Assistant assessment
GET	/api/profile/	User profile data

API Documentation View. *Purpose:* document available endpoints for consumers. *Design:* structured API browser/Swagger view. *Functionality:* enables testing and integration. *Business value:* accelerates frontend integration and future third-party use.

Error Handling and Versioning

Standardized error responses and a versioning-ready URL structure ensure forward compatibility and a smooth developer experience.

CHAPTER 12 — CLOUD DEPLOYMENT

12.1 Overview

The platform is deployed on **Google Cloud Run**, a fully managed serverless container platform. The application is containerized (Docker) and deployed as a stateless service that scales automatically with traffic.

Table 12.1 — Cloud Deployment Configuration

Aspect	Configuration
Platform	Google Cloud Run (asia-southeast1)
Packaging	Docker container
Scaling	Automatic, scale-to-zero capable
Database	PostgreSQL (managed/Cloud SQL compatible)
Networking	HTTPS endpoint, managed TLS
Live URL	https://nifty-100-intelligence-549099183935.asia-southeast1.run.app/

12.2 Deployment Pipeline

Source code → containerization → push to registry → deploy to Cloud Run → live HTTPS endpoint.

Cloud Deployment Pipeline. *Purpose:* illustrate build-to-deploy flow. *Design:* CI/CD-style pipeline diagram or Cloud Run console screenshot. *Functionality:* automated, repeatable deployment with managed scaling. *Business value:* low operational overhead, high availability, pay-per-use cost efficiency.

12.3 Benefits

- **Scalability:** automatic scaling under load.
 - **Cost efficiency:** pay only for usage.
 - **Reliability:** managed infrastructure and TLS.
 - **Region proximity:** hosted in **asia-southeast1** for low latency to Indian/Asian users.
-

CHAPTER 13 — DASHBOARD DEVELOPMENT

The platform includes four **Power BI** dashboards designed for distinct stakeholder needs. Each was built on the PostgreSQL warehouse's star schema using DAX measures and interactive visuals.

13.1 Dashboard 1 — Executive Overview Dashboard



Figure 13.1 — Executive Overview Dashboard. *Purpose:* deliver a high-level snapshot of the Nifty100 universe. *Design:* KPI cards (total market cap, average P/E, top movers), index-level trend charts, and sector contribution visuals. *Functionality:* interactive slicers by sector and period; drill-through to detail. *Business value:* gives decision-makers an instant pulse of market health and standout performers.

13.2 Dashboard 2 — Financial Trends Dashboard

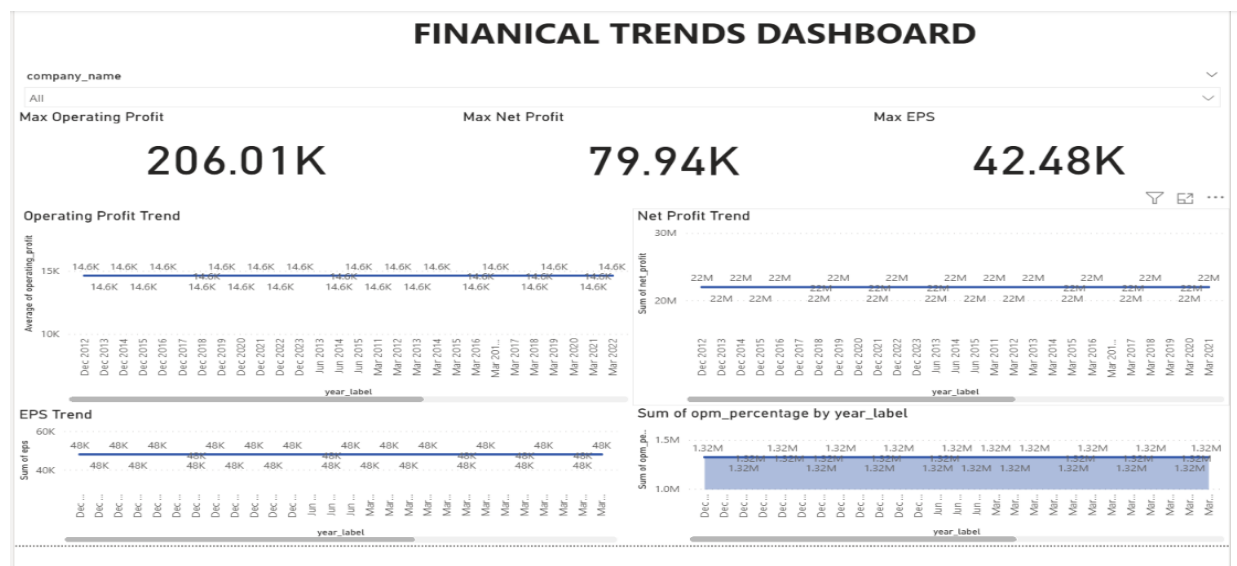


Figure 13.2 — Financial Trends Dashboard. *Purpose:* analyze financial performance over time. *Design:* time-series charts of revenue, profit, margins, and ratios; company comparison panels. *Functionality:* period-over-period trend analysis with filtering. *Business value:* reveals growth, profitability, and stability trends to support fundamental analysis.

13.3 Dashboard 3 — Sector Analytics Dashboard

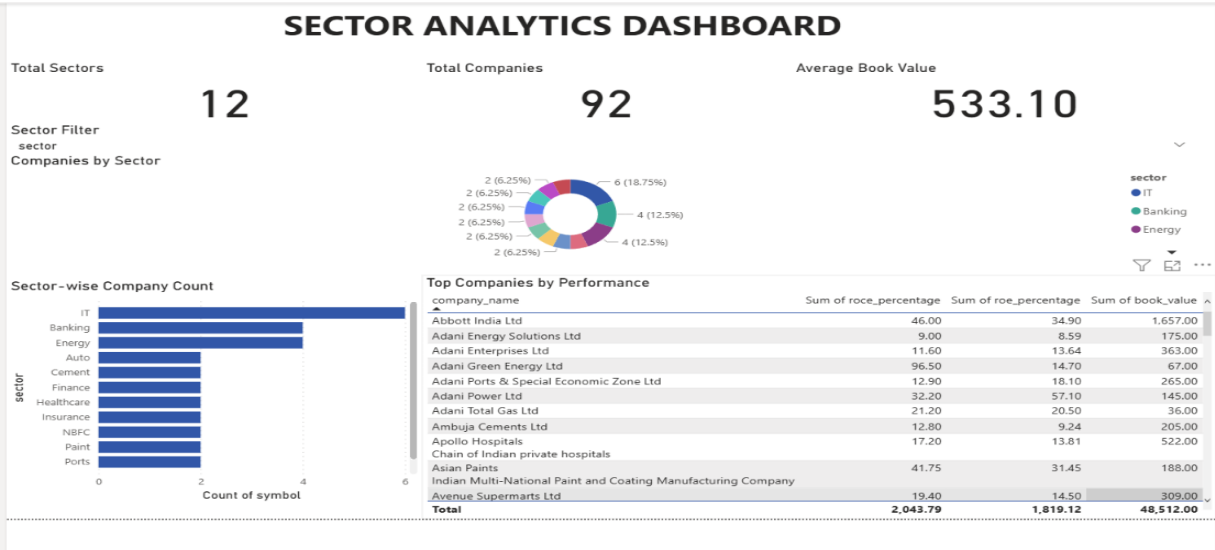


Figure 13.3 — Sector Analytics Dashboard. *Purpose:* compare and rank sectors. *Design:* sector heatmaps, contribution bars, and ratio comparisons across sectors. *Functionality:* identify leading and lagging sectors and rotation patterns. *Business value:* supports sector allocation and diversification strategies.

RY

13.4 Dashboard 4 — Investor Insights Dashboard

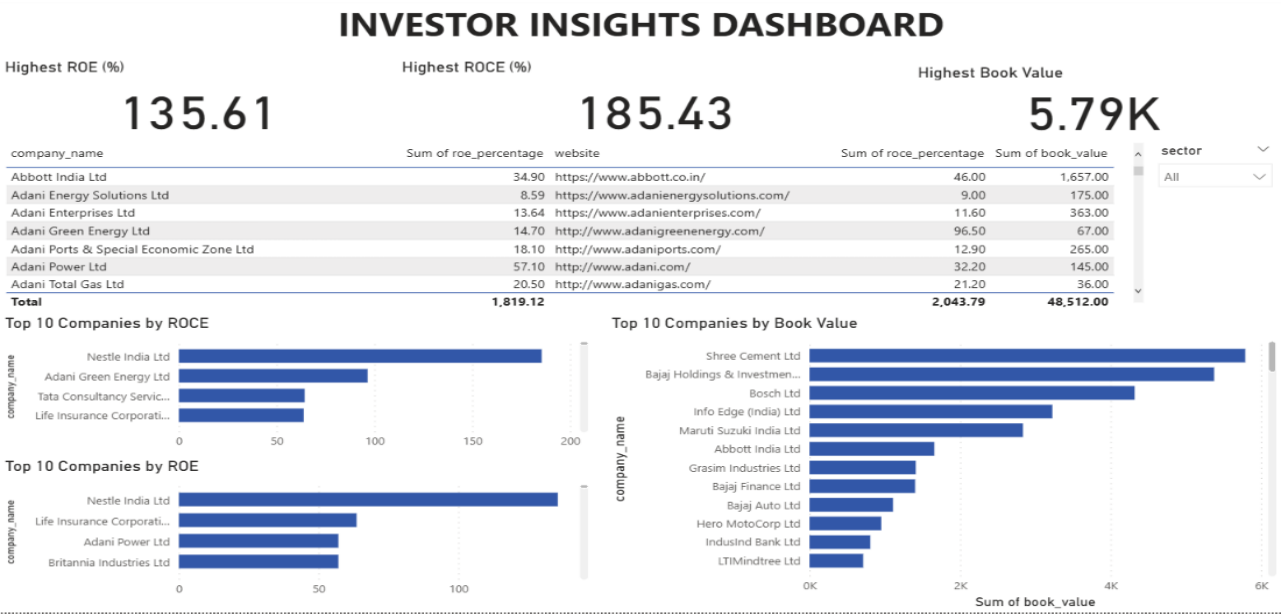


Figure 13.4 — Investor Insights Dashboard. *Purpose:* translate data into investor-centric signals. *Design:* valuation vs. quality scatter plots, risk indicators, dividend and return metrics. *Functionality:* highlight value, growth, and risk profiles. *Business value:* helps investors shortlist opportunities aligned to their strategy and risk appetite.

CHAPTER 14 — WEBSITE FEATURES

This chapter analyzes the core web application modules, each contributing to a cohesive financial intelligence experience.

14.1 Login System

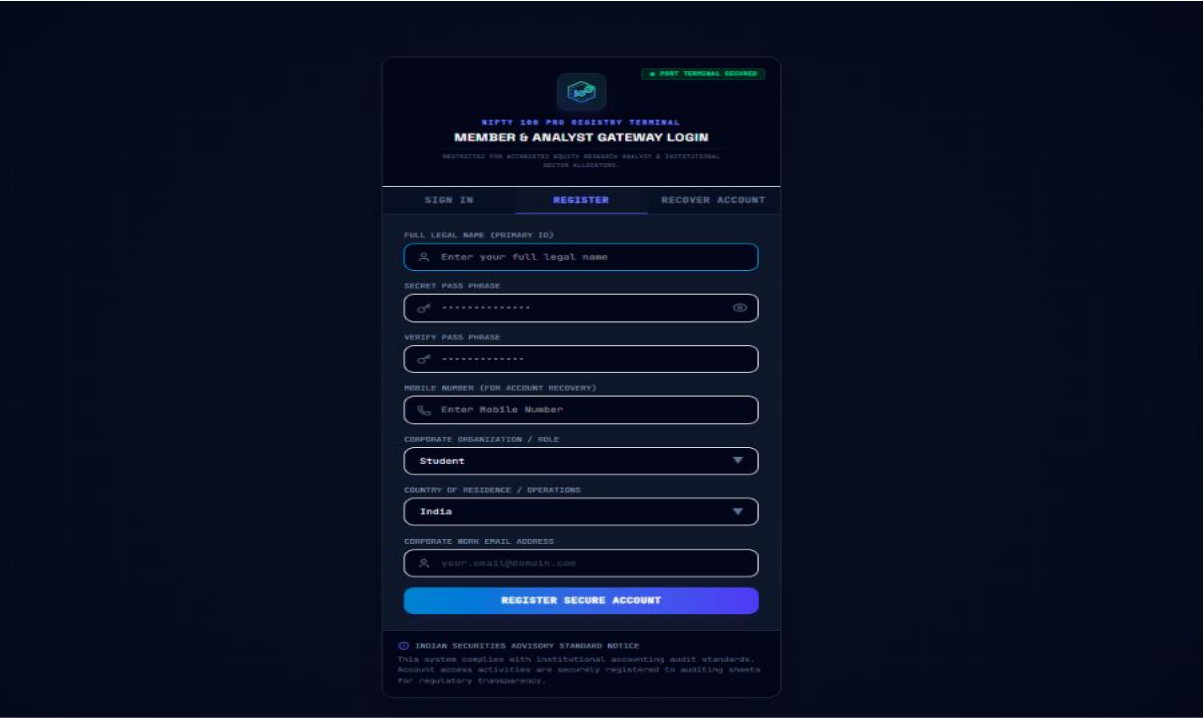
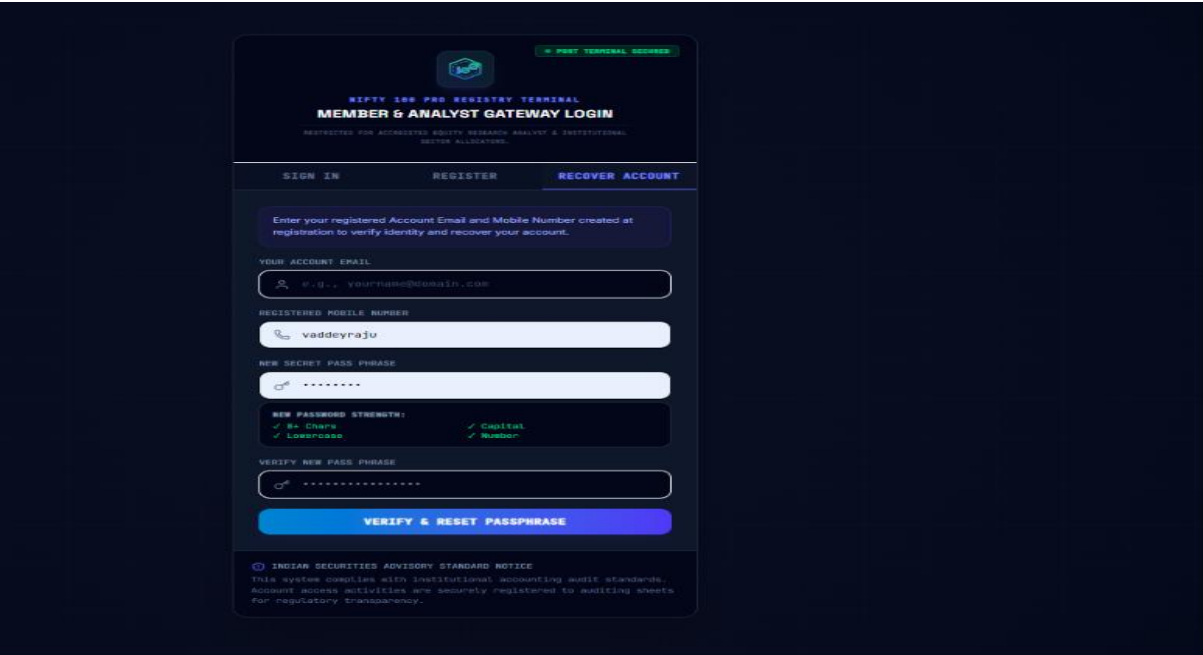


Figure 14.1 — Login System. *Purpose:* secure access control. *Design:* clean form with validation and error feedback. *Functionality:* token/session-based authentication against the Django backend. *Business value:* protects user data and enables personalization (watchlists, profiles, risk history)



14.2 User Profile Module

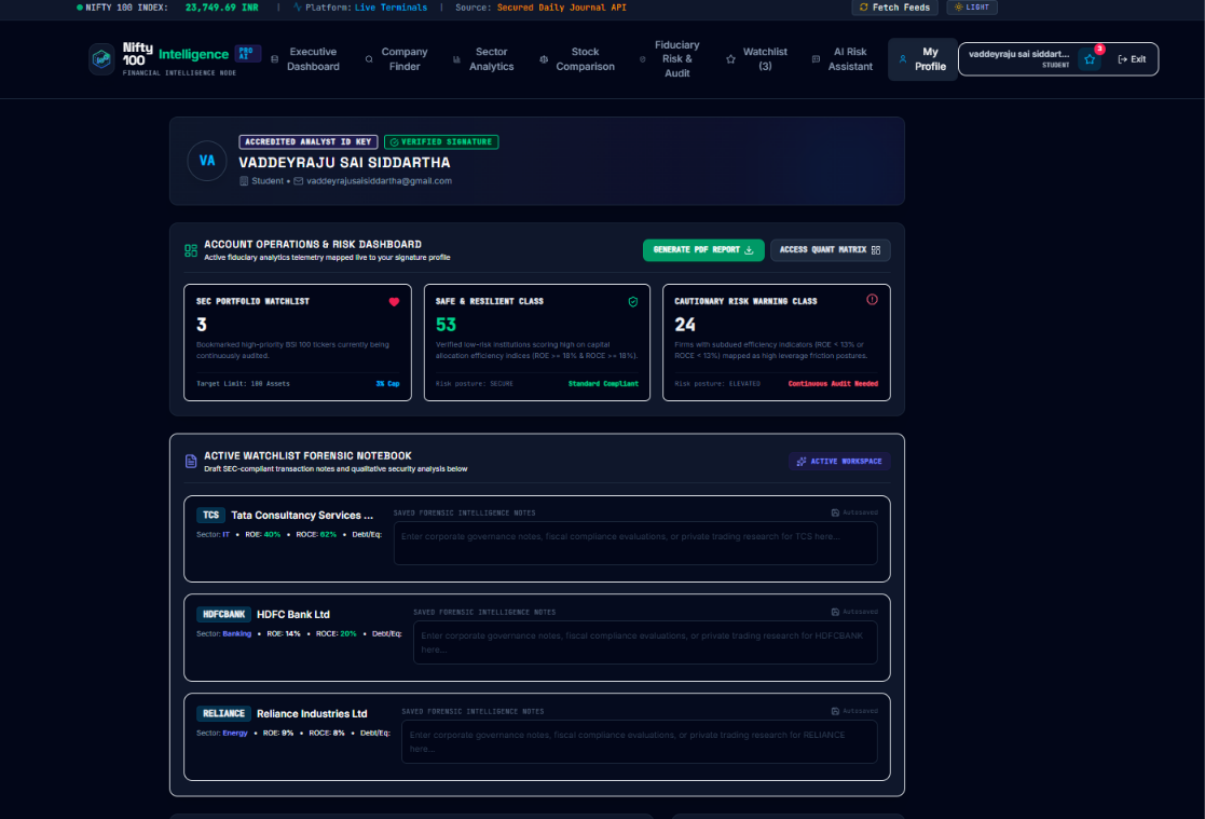
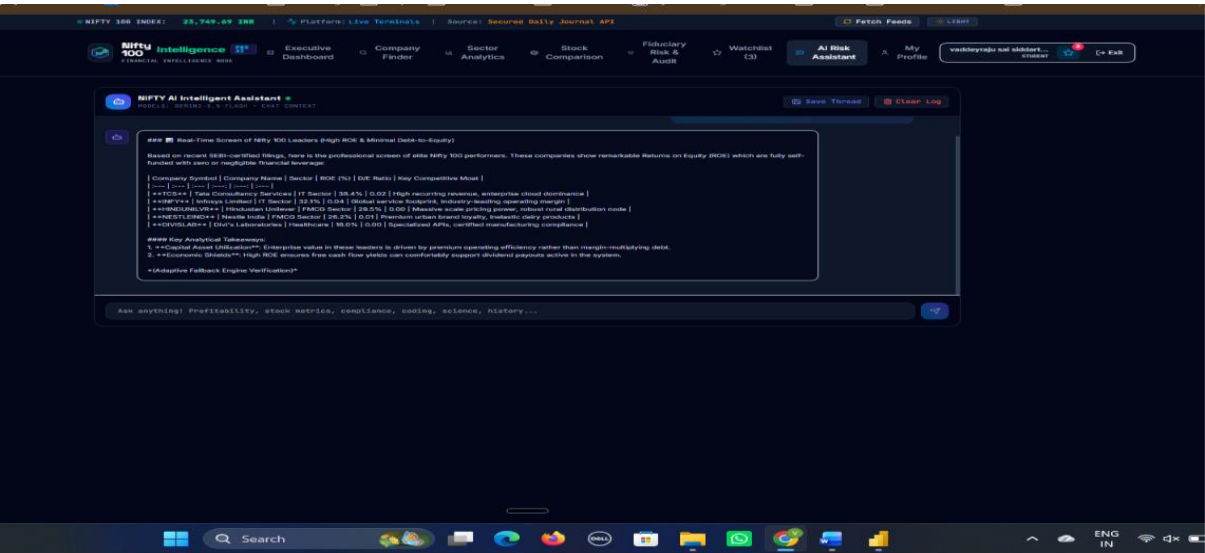


Figure 14.2 — User Profile. *Purpose:* manage user identity and preferences. *Design:* profile view with editable fields and risk preferences. *Functionality:* stores and updates profile and risk-appetite data. *Business value:* enables tailored insights and a personalized experience.

14.3 AI Risk Assistant



AI Risk Assistant. *Purpose:* guide users in interpreting risk. *Design:* conversational/assistive interface returning risk insights for companies. *Functionality:* analyzes financial indicators to surface risk signals and explanations. *Business value:* lowers the expertise barrier, helping non-experts make informed decisions.

14.4 Watchlist Module

[FIGURE 14.4 — INSERT SCREENSHOT: Watchlist Module]

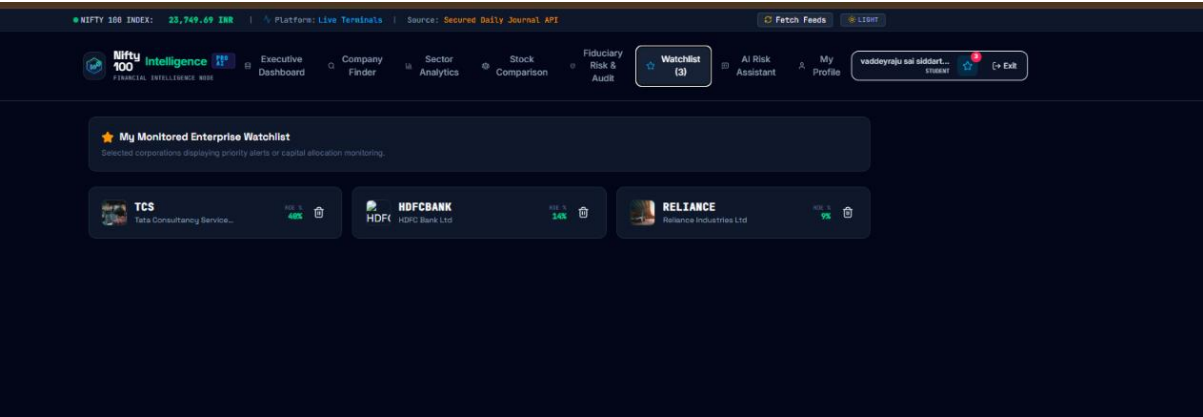


Figure 14.4 — Watchlist. *Purpose:* track companies of interest. *Design:* add/remove controls with a consolidated watchlist view. *Functionality:* persistent, user-specific watchlists via backend APIs. *Business value:* improves engagement and supports ongoing monitoring of investment candidates.

14.5 Company Finder

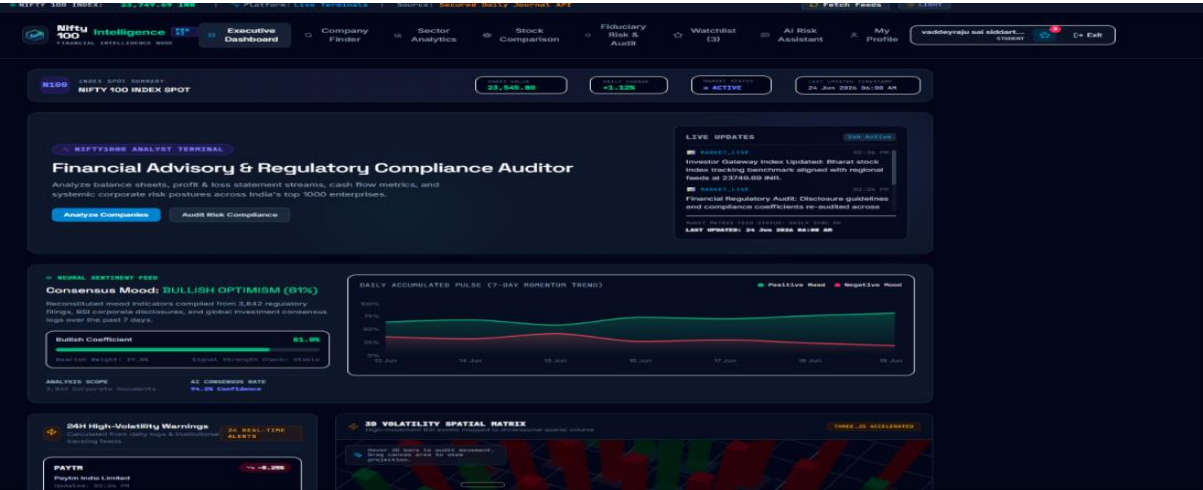
Company Finder. (See Figure 9.2.) *Purpose:* discover and filter companies. *Design:* searchable, filterable interface. *Functionality:* dynamic backend queries by name, sector, and metrics. *Business value:* speeds up research and comparison.

14.6 Sector Analytics

Sector Analytics (Web). *Purpose:* explore sector-level performance within the app. *Design:* sector cards/charts mirroring the Power BI sector view. *Functionality:* compares sector metrics interactively. *Business value:* informs diversification and allocation decisions directly in-app.

14.7 Financial Intelligence Dashboard (Web)

[FIGURE 14.5 — INSERT SCREENSHOT: Financial Intelligence Dashboard (Web)]



Financial Intelligence Dashboard. *Purpose:* unify key analytics in one in-app view. *Design:* combined KPI cards, charts, and tables. *Functionality:* aggregates company, financial, and sector intelligence. *Business value:* a single pane of glass for data-driven investment insight.

CHAPTER 15 — RESULTS AND FINDINGS

15.1 Business Insights

- The platform successfully consolidates the entire Nifty100 universe into one governed, queryable warehouse, eliminating data fragmentation.
- Market-capitalization concentration is visible across a handful of large sectors (notably financial services, IT, and energy), reflecting index composition realities.
- KPI dashboards enable rapid identification of top performers and laggards, reducing analysis time from hours to minutes.

15.2 Financial Insights

- Profitability and valuation metrics (P/E, ROE, margins) vary widely across companies, highlighting clear quality and value clusters.
- Trend analysis reveals which companies demonstrate consistent revenue and profit growth versus volatile performers.
- Leverage indicators (Debt-to-Equity) flag companies with elevated balance-sheet risk.

15.3 Sector Performance Insights

- Sector comparison reveals leading sectors by aggregate growth and profitability and identifies underperforming sectors.
- Sector rotation patterns become evident through comparative analytics, aiding allocation strategy.

15.4 Investor Analysis

- The Investor Insights dashboard and AI Risk Assistant together help classify companies along value/growth/risk dimensions.
- Personalized watchlists and risk profiles enable investors to focus on opportunities aligned with their strategy.

Table 15.1 — Summary of Key Findings

Dimension	Key Finding
Business	Unified Nifty100 data enables instant, comparative insight
Financial	Wide dispersion in valuation, profitability, and leverage
Sector	Clear leaders/laggards and rotation signals
Investor	Effective value/growth/risk segmentation and personalization

CHAPTER 16 — ADVANTAGES

End-to-end integration

from raw data to insight in one platform.

Cloud-native scalability

via Google Cloud Run with low operational cost.

Rich business intelligence

through four purpose-built Power BI dashboards.

Intelligent assistance

via the AI Risk Assistant, lowering the expertise barrier.

Personalization

through profiles and watchlists.

Clean, governed data

ensured by robust ETL and validation.

Modular, maintainable architecture

supporting future growth.

Accessibility

— always-available web app reachable from any device.

CHAPTER 17 — LIMITATIONS

Scope limited to Nifty100

— does not yet cover broader indices or mid/small-caps.

Batch data refresh

— not real-time/intraday tick data.

AI Risk Assistant

provides guidance, not regulated financial advice.

Dependency on data source quality

— insights are only as accurate as ingested data.

No brokerage integration

— the platform informs but does not execute trades.

- **Historical depth**
- constrained by available data and project timeline.

CHAPTER 18 — FUTURE ENHANCEMENTS

Expand coverage

to Nifty500, mid-cap, and small-cap universes.

Real-time data streaming

for intraday analytics.

Advanced ML models

for predictive analytics and richer risk scoring.

Portfolio management

with simulated/paper trading and P&L tracking.

Alerts and notifications

(price/ratio thresholds, watchlist events).

Mobile applications

(iOS/Android) for on-the-go access.

Broker API integration

for end-to-end investing workflows.

Natural-language querying

("show me low-debt IT companies under P/E 25").

Role-based access

for institutional/team use.

CHAPTER 19 — CONCLUSION

The **Nifty100 Financial Intelligence Platform** successfully delivers a production-grade, cloud-deployed financial analytics ecosystem that unifies data engineering, full-stack development, business intelligence, and cloud operations. Over the internship period at **BlueStock Fintech (22 April – 22 June 2026)**, the team designed and implemented robust ETL pipelines, a PostgreSQL analytical warehouse, a secure Django REST API layer, a modern React.js web application with intelligent and personalized features, four Power BI dashboards, and a scalable Google Cloud Run deployment.

The project demonstrates how modern engineering practices can democratize sophisticated financial intelligence for investors, analysts, and researchers. Beyond the technical artifacts, the internship provided invaluable experience in industry workflows — agile collaboration, version control, code review, modular ownership, and end-to-end delivery. The platform stands as a strong foundation for the future enhancements outlined, and validates the team's ability to translate a real-world fintech problem into a deployed, value-generating product.

REFERENCES

1. National Stock Exchange of India (NSE) — Nifty100 Index Methodology and Constituents. <https://www.nseindia.com>
 2. Django Software Foundation — *Django Documentation*. <https://docs.djangoproject.com>
 3. Django REST Framework — *Official Documentation*. <https://www.django-rest-framework.org>
 4. React.js — *Official Documentation*. <https://react.dev>
 5. PostgreSQL Global Development Group — *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
 6. Microsoft — *Power BI Documentation*. <https://learn.microsoft.com/power-bi/>
 7. Google Cloud — *Cloud Run Documentation*. <https://cloud.google.com/run/docs>
 8. Kimball, R., & Ross, M. *The Data Warehouse Toolkit* — Dimensional Modeling.
 9. Python Software Foundation — *Python Documentation*. <https://docs.python.org>
 10. pandas — *Data Analysis Library Documentation*. <https://pandas.pydata.org>
 11. Project Repository — <https://github.com/VaddeyrajuSaisiddartha/Nifty100-Financial-Intelligence-Platform>
 12. Live Application — <https://nifty-100-intelligence-549099183935.asia-southeast1.run.app/>
-

APPENDIX

Appendix A — Project Links

- GitHub: <https://github.com/VaddeyrajuSaisiddartha/Nifty100-Financial-Intelligence-Platform>
- Live App: <https://nifty-100-intelligence-549099183935.asia-southeast1.run.app/>

Appendix B — Technology Stack Summary

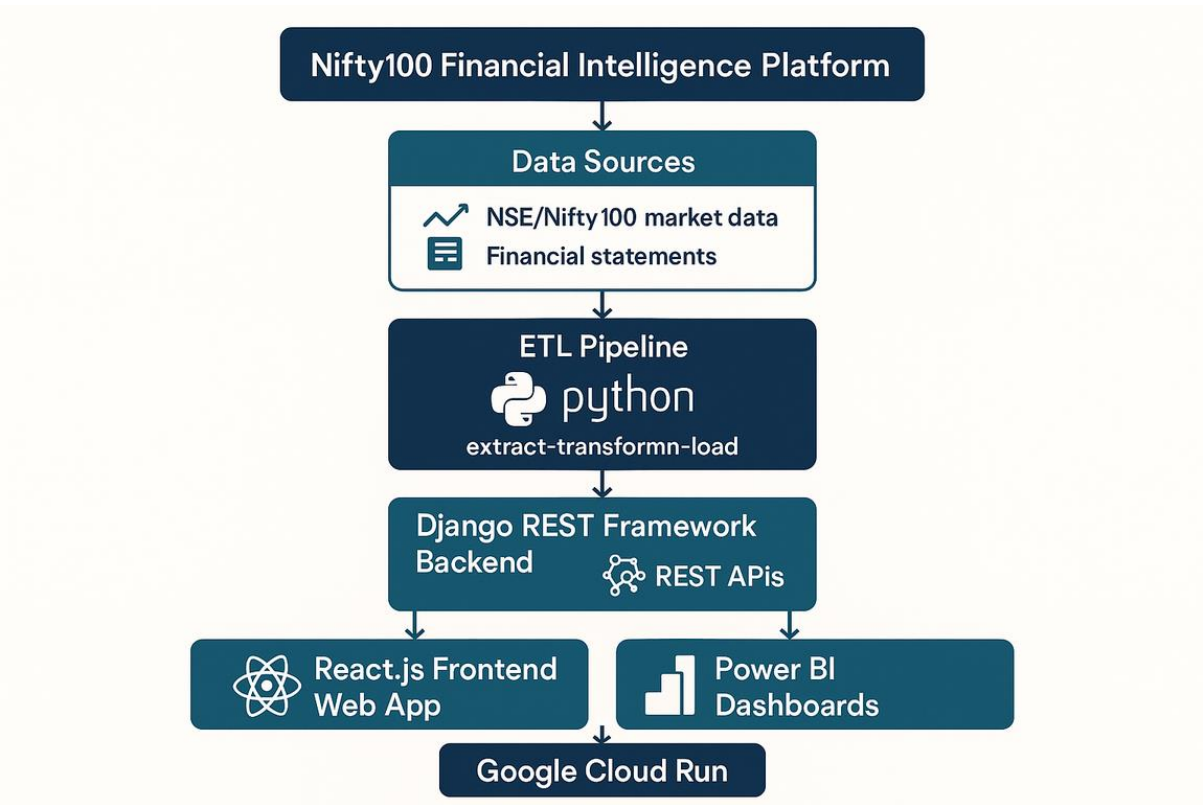
React.js · Django REST Framework · PostgreSQL · Python · Power BI · ETL Pipelines · Google Cloud Run · Docker

Appendix C — Team & Responsibilities

Member	Role	Modules
Vaddeyraju Sai Siddartha	Team Lead	Architecture, Backend, Deployment
Supriya Sinha	Member	Frontend / React UI
Rohit R	Member	ETL & Database
Adi Singh	Member	Power BI Dashboards

Let me check on the architecture diagram and then provide the slide deck.

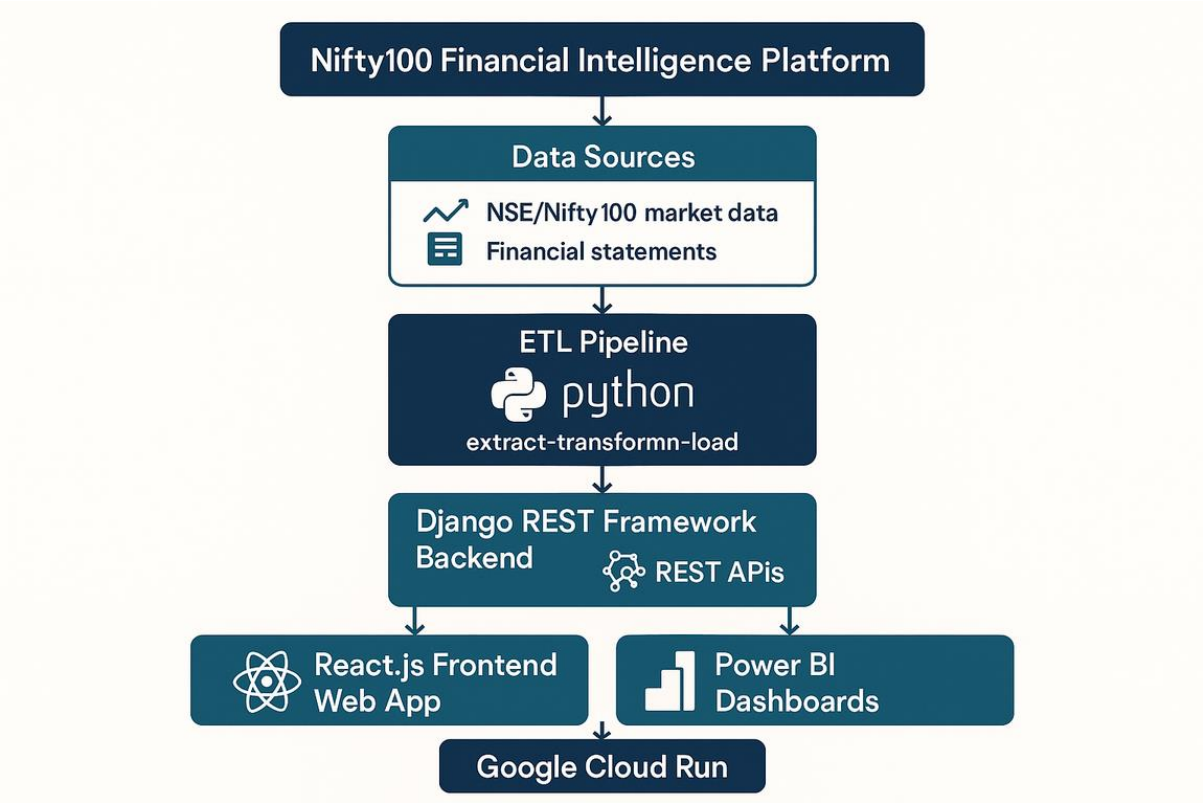
Ran image_get_image_async



I have the architecture diagram. Now let me create the title slide background image, and then I'll deliver the complete report and presentation content.

Ran image_create_image_async

Ran image_get_image_async



The architecture diagram is ready. Let me check the title slide background and generate the dashboard mockups.

Ran image_get_image_async

Let me retrieve the architecture diagram result and continue.

Ran image_get_image_async

